

Ordinamento dei dati

Ordinare i dati è una attività semplice e sovente necessaria per la quale **R** offre molte soluzioni.

Qui vediamo le funzioni di ordinamento contenute nel *pacchetto base* di **R** - il pacchetto di funzioni statistiche e grafiche che viene installato con il programma **R** - e le funzioni contenute nei due *pacchetti aggiuntivi* **dplyr** e **data.table**, che sono quelle più frequentemente utilizzate. I due pacchetti aggiuntivi devono ovviamente essere scaricati e installati dal **CRAN**.

Per proseguire è necessario:

→ effettuare il download del file di dati `peso_altezza.csv`

→ salvare il file nella cartella `C:\Rdati\`

Per il file di dati trovate link e modalità di download alla [pagina Dati](#), ma potete anche semplicemente copiare i dati riportati qui sotto aggiungendo un ↵ Invio al termine dell'ultima riga e salvarli in `C:\Rdati\` in un file di testo denominato `peso_altezza.csv` (assicuratevi che il file sia effettivamente salvato in formato testo e con l'estensione `.csv`).

```
id;sezzo;anni;peso_kg;altezza_m
MT;M;69;76;1,78
GF;F;56;63;
MC;F;53;71;1,60
SB;M;28;73;1,78
FE;F;61;54;1,54
AB;M;46;92;1,84
RF;F;31;81;1,56
```

Iniziamo con la funzione **order()** contenuta nel pacchetto base precisando subito che con questa funzione la tabella originaria viene sì mostrata nell'ordine desiderato ma non viene modificata, e il nuovo ordine si applica salvando i dati ordinati in una tabella che:

→ può avere lo stesso nome della tabella originaria, nel qual caso questa viene sovrascritta e quindi risulterà effettivamente modificata;

→ può avere un nuovo nome, nel qual caso la tabella originaria resta immodificata, come facciamo qui (noi che vogliamo essere prudenti).

Vediamo la cosa con questo semplice script, incollatelo nella `Console di R` e premete ↵ Invio:

```
# ORDINAMENTO DATI
#
mydata <- read.table("c:/Rdati/peso_altezza.csv", header=TRUE, sep=";", dec=",") #
importa i dati
mydata # mostra la tabella originaria
#
mydata[order(mydata$id),] # ordina e mostra la tabella ordinata
#
mydata # la tabella originaria resta immodificata
#
newdata <- mydata[order(mydata$id),] # ordina e salva in una nuova tabella
#
newdata # mostra la nuova tabella
#
```

Ecco quindi cosa accade, questa è la tabella originaria **mydata** importata:

```
> mydata # mostra la tabella originaria
```

	id	sex	age	weight_kg	height_m
1	MT	M	69	76	1.78
2	GF	F	56	63	NA
3	MC	F	53	71	1.60
4	SB	M	28	73	1.78
5	FE	F	61	54	1.54
6	AB	M	46	92	1.84
7	RF	F	31	81	1.56

Con la funzione **order()** la tabella **mydata** viene ordinata in ordine crescente in base alla variabile **id** e viene mostrata nella Console di R:

```
> mydata[order(mydata$id),] # ordina e mostra la tabella ordinata
```

	id	sex	age	weight_kg	height_m
6	AB	M	46	92	1.84
5	FE	F	61	54	1.54
2	GF	F	56	63	NA
3	MC	F	53	71	1.60
1	MT	M	69	76	1.78
7	RF	F	31	81	1.56
4	SB	M	28	73	1.78

Tuttavia se verifichiamo la tabella originaria **mydata** richiamandone il nome vediamo che non è stata modificata:

```
> mydata # la tabella originaria resta immodificata
```

	id	sex	age	weight_kg	height_m
1	MT	M	69	76	1.78
2	GF	F	56	63	NA
3	MC	F	53	71	1.60
4	SB	M	28	73	1.78
5	FE	F	61	54	1.54
6	AB	M	46	92	1.84
7	RF	F	31	81	1.56

Per ottenere una tabella con i dati ordinati si ordina nuovamente la tabella **mydata** salvando il risultato (**<-**) in una nuova tabella che denominiamo **newdata**:

```
> newdata <- mydata[order(mydata$id),] # ordina e salva in una nuova tabella
```

Se ora verifichiamo il contenuto della nuova tabella **newdata** richiamandone il nome, vediamo che contiene i dati ordinati:

```
> newdata # mostra la nuova tabella
```

	id	sex	age	weight_kg	height_m
6	AB	M	46	92	1.84
5	FE	F	61	54	1.54
2	GF	F	56	63	NA
3	MC	F	53	71	1.60
1	MT	M	69	76	1.78
7	RF	F	31	81	1.56
4	SB	M	28	73	1.78

Una volta chiarito questo punto, molto importante dal punto di vista pratico, possiamo procedere con gli script. Copiate il primo, incollatelo nella Console di R e premete ↵ Invio:

```

# ORDINAMENTO ordina per nome della tabella$variabile
#
mydata <- read.table("c:/Rdati/peso_altezza.csv", header=TRUE, sep=";", dec=",") #
importa i dati
mydata # mostra la tabella originaria
#
mydata[order(mydata$peso_kg),] # ordina il peso in ordine crescente
#
mydata[order(-mydata$peso_kg),] # ordina il peso in ordine decrescente
#
mydata[order(mydata$sex, -mydata$peso_kg),] # ordina il sesso in ordine crescente, il peso
in ordine decrescente
#
mydata[order(mydata$id),] # ordina id in ordine crescente
#
mydata[order(-mydata$id),] # ordina id in ordine decrescente
#
mydata[order(-xtfrm(mydata$id)),] # ordina id in ordine decrescente
#
mydata[order(-xtfrm(mydata$sex), mydata$peso_kg),] # ordina il sesso in ordine
decrescente, il peso in ordine crescente
#
mydata[order(mydata$altezza_m, na.last=TRUE),] # ordina l'altezza in ordine crescente, NA in
coda
#
mydata[order(mydata$altezza_m, na.last=FALSE),] # ordina l'altezza in ordine crescente, NA
in testa
#
mydata[order(-mydata$altezza_m, na.last=NA),] # ordina l'altezza in ordine decrescente,
esclude NA
#
mydata # la tabella originaria resta immodificata
#

```

I commenti riportati in ciascuna riga di codice sono di per sè esplicativi.

La cosa interessante accade quando si tenta di ordinare la variabile **mydata\$id** in ordine decrescente con **mydata[order(-mydata\$id),]**, questo infatti è il risultato:

```

> mydata[order(-mydata$id),] # ordina id in ordine decrescente
Errore in -mydata$id : argomento non valido per l'operatore unario

```

L'errore è legato al fatto che **R** non ammette davanti a una variabile di tipo testo il segno meno (-) e viene corretto nella riga successiva con la funzione **xtfrm()** che nell'help è definita come "A generic auxiliary function that produces a numeric vector which will sort in the same order as x".

Questo è il risultato della nuova sintassi che corregge l'errore che impediva l'ordinamento in senso decrescente della variabile testo:

```

> mydata[order(-xtfrm(mydata$id)),] # ordina id in ordine decrescente
  id sesso anni peso_kg altezza_m
4 SB     M   28     73     1.78
7 RF     F   31     81     1.56
1 MT     M   69     76     1.78
3 MC     F   53     71     1.60

```

2	GF	F	56	63	NA
5	FE	F	61	54	1.54
6	AB	M	46	92	1.84

Come potete constatare dai risultati che vedete nella `Console di R` le ultime tre righe di codice illustrano come impiegare l'argomento **na.last** per controllare il trattamento dei dati mancanti (NA):

- se **na.last=TRUE** i dati mancanti sono identificati con NA e messi in coda ai dati;
- se **na.last=FALSE** i dati mancanti sono identificati con NA e posti in testa ai dati;
- se **na.last=NA** i dati mancanti sono rimossi;

Copiate questo secondo script, incollatelo nella `Console di R` e premete ↵ Invio:

```
# ORDINAMENTO ordina per nome della variabile
#
mydata <- read.table("c:/Rdati/peso_altezza.csv", header=TRUE, sep=";", dec=",") #
importa i dati
mydata # mostra la tabella originaria
attach(mydata) # consente di impiegare direttamente i nomi delle variabili di mydata
#
mydata[order(id),] # ordina id in ordine crescente
#
mydata[order(-xtfrm(id)),] # ordina id in ordine decrescente
#
mydata[order(-anni),] # ordina gli anni in ordine decrescente
#
mydata[order(sesso, -altezza_m),] # ordina il sesso in ordine crescente, l'altezza in ordine
crescente
#
mydata[order(-xtfrm(sesso), altezza_m),] # ordina il sesso in ordine decrescente, l'altezza in
ordine decrescente
#
detach(mydata) # termina l'impiego diretto dei nomi delle variabili
#
mydata # la tabella originaria resta immodificata
#
```

Impieghiamo sempre la funzione **order()** contenuta nel pacchetto `{base}` ma risulta immediatamente evidente la differenza: nello script precedente le variabili erano indicate con *nomedellatabella\$nomedellavariabile*, mentre ora dopo avere riportato nella seconda riga di codice

attach(mydata)

possiamo fare riferimento alle variabili riportando semplicemente il *nomedellavariabile*. Mentre le regole di ordinamento rimangono ovviamente le stesse, alla fine dello script viene riportato

detach(mydata)

per ripristinare la condizione di default [2].

Concludiamo quanto riguarda la funzione **order()** contenuta nel pacchetto `{base}` con questo terzo script, incollatelo nella `Console di R` e premete ↵ Invio:

```
# ORDINAMENTO ordina per numero della colonna
#
```

```

mydata <- read.table("c:/Rdati/peso_altezza.csv", header=TRUE, sep=";", dec=",") #
importa i dati
mydata # mostra la tabella originaria
#
mydata[order(mydata[,4]),] # ordina la colonna 4 in ordine crescente
#
mydata[order(-mydata[,4]),] # ordina la colonna 4 in ordine decrescente
#
mydata[order(mydata[,1]),] # ordina la colonna 1 in ordine crescente
#
mydata[order(-xtfrm(mydata[,1])),] # ordina la colonna 1 in ordine decrescente
#
mydata[order(mydata[,2], mydata[,5]),] # ordina la colonna 2 in ordine crescente, la colonna 5
in ordine crescente
#
mydata[order(-xtfrm(mydata[,2]), -mydata[,5]),] # ordina la colonna 2 in ordine decrescente,
la colonna 5 in ordine decrescente
#
mydata # la tabella originaria resta imm modificata
#

```

La differenza rispetto ai due primi script consiste, questa volta, semplicemente nell'ordinare la tabella impiegando il numero della colonna.

Vediamo ora le regole per l'ordinamento previste dalla funzione **arrange()** del pacchetto **dplyr**.

Copiate questo script, incollatelo nella `Console di R` e premete ↵ Invio:

```

# ORDINAMENTO con il pacchetto "dplyr"
#
library(dplyr) # carica il pacchetto
mydata <- read.table("c:/Rdati/peso_altezza.csv", header=TRUE, sep=";", dec=",") #
importa i dati
mydata # mostra la tabella originaria
#
arrange(mydata, altezza_m) # ordina l'altezza in ordine crescente
#
arrange(mydata, desc(altezza_m)) # ordina l'altezza in ordine decrescente
#
arrange(mydata, sesso, desc(id)) # ordina il sesso in ordine crescente, id in ordine decrescente
#
mydata # la tabella originaria resta imm modificata
#

```

Le cose da notare in merito alla funzione **arrange()** sono:

- viene specificato ogni volta il nome della tabella da ordinare, per cui il nome della variabile risulta abbreviato;
- i dati mancanti sono identificati con `NA` e messi in coda ai dati;
- l'ordinamento di una variabile testo in ordine discendente con l'opzione **desc()** non incontra il problema che avevamo riscontrato con la funzione **order()**;
- la tabella originaria resta imm modificata.

Passiamo infine a vedere le regole per l'ordinamento previste dalla funzione **setorder()** del pacchetto **read.table**.

Copiate questo script, incollatelo nella `Console di R` e premete ↵ Invio:

```
# ORDINAMENTO con il pacchetto "data.table"
#
library(data.table) # carica il pacchetto
mydata <- read.table("c:/Rdati/peso_altezza.csv", header=TRUE, sep=";", dec=",") #
importa i dati
mydata # mostra la tabella originaria
#
setorder(mydata, altezza_m) # ordina l'altezza in ordine crescente
mydata # mostra la tabella ordinata
#
setorder(mydata, -altezza_m) # ordina l'altezza in ordine decrescente
mydata # mostra la tabella ordinata
#
setorder(mydata, sesso, -id) # ordina il sesso in ordine crescente, id in ordine decrescente
mydata # mostra la tabella ordinata
#
```

Qui le cose da notare sono:

→ contrariamente a quanto visto finora, con la funzione **setorder()** la tabella originaria viene realmente ordinata e inoltre:

→ se si impiega l'argomento **na.last=TRUE** i dati mancanti sono identificati con **NA** e messi in coda ai dati;

→ se si impiega l'argomento **na.last=FALSE** i dati mancanti sono identificati con **NA** e posti in testa ai dati;

→ la funzione accetta per l'argomento **na.last** solamente i valori **TRUE** e **FALSE** con valore di default **FALSE** (che è quello qui esemplificato).

Conclusione: se volete evitare l'impiego di un pacchetto aggiuntivo, trovate nella funzione **order()** del pacchetto {base} che viene installato con il programma **R** tutte le modalità di ordinamento che si possono desiderare, anche se con qualche arzigogolo. Se invece vi va bene installare pacchetti aggiuntivi, la funzione **arrange()** e la funzione **setorder()**, con differenze veramente minime tra loro, offrono il vantaggio di impiegare una sintassi semplificata e più immediata. Il punto delicato è il fatto che **setorder()** ordina realmente la tabella originaria, mentre potrebbe essere più prudente e anche più ordinato lasciare, come accade con **arrange()**, la tabella originaria immodificata e al bisogno salvare i dati ordinati in una tabella con un nome differente. Ma ovviamente si tratta di scelte personali e insindacabili.

[1] Parliamo di **array** o **vettore** nel caso di dati numerici monodimensionali, disposti su una sola riga,

8	6	11	7
---	---	----	---

di **matrice** nel caso di **dati numerici** disposti su più righe e più colonne

8	9	15	14
6	7	18	12
11	8	17	13
7	4	19	17

e di **tabella** nei casi in cui il contenuto, disposto su più righe e più colonne, è rappresentato oltre che da **dati numerici**, anche da **testo** e/o **operatori logici**

M	7	9	VERO
F	3	12	VERO
F	5	10	FALSO

[2] In termini un poco più tecnici: con la funzione **attach(database)** il database è collegato al percorso di ricerca di **R**, questo significa che quando si opera su una variabile **R** è informato del database che la contiene, e pertanto è possibile accedere alla variabile semplicemente fornendone il nome. Con la funzione **detach(database)** il database viene rimosso dal percorso di ricerca di **R**.
